

Formal Methods and Specification (LS 2025/26)

Lecture 8: Termination, Total Program Correctness

Stefan Ratschan

Czech Technical University in Prague



Evropský sociální fond Praha & EU: Investujeme do vaší budoucnosti

Collatz Problem, Decidability

```
assume  $x \geq 1$   
while  $x \neq 1$  do  
  if  $x \bmod 2 = 0$  then  
     $x \leftarrow x/2$   
  else  
     $x \leftarrow 3x + 1$ 
```

Terminates for every input x ?

Theorem [Turing, 1937]:

The problem of checking **termination** (halting problem) is **undecidable**.

Today's Lecture

How to prove that a certain **program terminates**?

- ▶ formal foundations
- ▶ manual methods
- ▶ (partially) automatic methods

At the same time we practice working with assertions

Basic Definitions

A partially correct program P is *terminating* iff
every regular execution of P
reaches a program line with the statement **return**.

Implicit assumption:
a **return** statement after the very last program line

Attention: Termination does **not** imply that
there is a constant **upper bound** on the number of loop iterations

Example?

```
 $i \leftarrow 0$   
while  $i < n$  do  
   $i \leftarrow i + 1$ 
```

Partial vs. Total Correctness

```
assume  $x = 564$   
while  $\top$  do  
     $@ x = 564$   
 $@ x = 564$   
return  $x$ 
```

The **assertions hold** on all executions, i.e., the program is *partially correct*.

We can prove that. The verification conditions are:

- ▶ $[x = 564 \wedge \top] \Rightarrow x = 564$ (twice)
- ▶ $[x = 564 \wedge \neg \top] \Rightarrow x = 564$ (twice)

But: The program is not **terminating**.

We want *total correctness* (=partial correctness + termination).

Well-Founded Relations

A relation $\prec \subseteq \Omega \times \Omega$ is a *well-founded* on $S \subseteq \Omega$ iff
there is no infinite sequence $s_1 \in S, s_2 \in S, \dots$ such that
 $s_1 \succ s_2 \succ \dots$

Example: $<$ is well founded on the natural numbers

$$5 > 4 > 3 > 2 > 1$$

but not on $\mathbb{R}^{\geq 0}$:

$$1 > \frac{1}{2} > \frac{1}{4} > \frac{1}{8} \dots$$

Usually based on a strict partial order

Example: Searching in a Sorted Array

For $a \in \mathcal{A}$, $n \in \mathcal{N}$, $\text{sorted}(a, n) :\Leftrightarrow \forall i \in \{1, \dots, n-1\} . a[i] \leq a[i+1]$

assume $a \in \mathcal{A}$, $\text{sorted}(a, n)$, $x \in \mathcal{N}$, $\exists i \in \{1, \dots, n\} . a[i] = x$

$l \leftarrow 1$; $u \leftarrow n$

while $a[(l+u)/2] \neq x$ **do**

 @ $\exists i \in \{l, \dots, u\} . a[i] = x$

if $a[(l+u)/2] < x$ **then** $l \leftarrow (l+u)/2 + 1$

else $u \leftarrow (l+u)/2 - 1$

@ $a[(l+u)/2] = x$

$r \leftarrow (l+u)/2$

@ $a[r] = x$

return r

Example: Searching in a Sorted Array

```
assume  $a \in \mathcal{A}$ ,  $\text{sorted}(a, n)$ ,  $x \in \mathcal{N}$ ,  $\exists i \in \{1, \dots, n\} . a[i] = x$   
 $l \leftarrow 1$ ;  $u \leftarrow n$   
while  $a[(l + u)/2] \neq x$  do  
   $\textcircled{\text{O}}$   $\exists i \in \{l, \dots, u\} . a[i] = x$   
  if  $a[(l + u)/2] < x$  then  $l \leftarrow (l + u)/2 + 1$   
  else  $u \leftarrow (l + u)/2 - 1$   
 $\textcircled{\text{O}}$   $a[(l + u)/2] = x$   
 $r \leftarrow (l + u)/2$   
 $\textcircled{\text{O}}$   $a[r] = x$   
return  $r$ 
```

Basic path (loop iteration, **if** branch):

```
assume  $\exists i \in \{l, \dots, u\} . a[i] = x$   
assume  $a[(l + u)/2] < x$   
 $l \leftarrow (l + u)/2 + 1$   
assume  $a[(l + u)/2] \neq x$   
 $\textcircled{\text{O}}$   $\exists i \in \{l, \dots, u\} . a[i] = x$ 
```

Verification Condition

Basic path:

assume $\exists i \in \{l, \dots, u\} . a[i] = x$

assume $a[(l + u)/2] < x$

$l \leftarrow (l + u)/2 + 1$

assume $a[(l + u)/2] \neq x$

@ $\exists i \in \{l, \dots, u\} . a[i] = x$

$$\left[\begin{array}{l} \exists i \in \{l, \dots, u\} . a[i] = x \wedge \\ a[(l + u)/2] < x \wedge \\ l_1 = (l + u)/2 + 1 \wedge \\ a[(l_1 + u)/2] \neq x \end{array} \right] \Rightarrow \exists i \in \{l_1, \dots, u\} . a[i] = x$$

Holds?

No, only under assumption $\text{sorted}(a, n)$

Program Termination

```
assume  $a \in \mathcal{A}$ ,  $n \in \mathcal{N}$ ,  $\text{sorted}(a, n)$ ,  $x \in \mathcal{N}$ ,  $\exists i \in \{1, \dots, n\} . a[i] = x$   
 $l \leftarrow 1$ ;  $u \leftarrow n$   
 $\textcircled{\text{O}} \exists i \in \{l, \dots, u\} . a[i] = x$   
while  $a[(l + u)/2] \neq x$  do  
     $\textcircled{\text{O}} \exists i \in \{l, \dots, u\} . a[i] = x$   
    if  $a[(l + u)/2] < x$  then  $l \leftarrow (l + u)/2 + 1$   
    else  $u \leftarrow (l + u)/2 - 1$   
 $\textcircled{\text{O}} a[(l + u)/2] = x$   
 $r \leftarrow (l + u)/2$   
 $\textcircled{\text{O}} a[r] = x$   
return  $r$ 
```

Observation:

A program is terminating iff **each loop** has only finitely many iterations

Remaining question: termination of loops

Loop Termination: Assertions

$v \leftarrow \infty$

while $a[(l + u)/2] \neq x$ **do**

 @ $u - l < v \wedge u - l \geq 1 \wedge \exists i \in \{l, \dots, u\} . a[i] = x$

$v \leftarrow u - l$

if $a[(l + u)/2] < x$ **then** $l \leftarrow (l + u)/2 + 1$

else $u \leftarrow (l + u)/2 - 1$

How to ensure that the loop terminates?

$u - l$ decreases, but always $u - l \geq 1$,

$<$ is well founded on the natural numbers

Loop Termination: Proof

$v \leftarrow \infty$

while $a[(l + u)/2] \neq x$ **do**

$@ u - l < v \wedge u - l \geq 1 \wedge \exists i \in \{l, \dots, u\} . a[i] = x$

$v \leftarrow u - l$

if $a[(l + u)/2] < x$ **then** $l \leftarrow (l + u)/2 + 1$

else $u \leftarrow (l + u)/2 - 1$

Basic path, first branch of **if-then-else**:

assume $u - l < v \wedge u - l \geq 1 \wedge \exists i \in \{l, \dots, u\} . a[i] = x$

$v \leftarrow u - l$

assume $a[(l + u)/2] < x$

$l \leftarrow (l + u)/2 + 1$

assume $a[(l + u)/2] \neq x$ **do**

$@ u - l < v \wedge u - l \geq 1 \wedge \exists i \in \{l, \dots, u\} . a[i] = x$

Loop Termination: Proof

Basic path, first branch of **if-then-else**:

assume $u - l < v \wedge u - l \geq 1 \wedge \exists i \in \{l, \dots, u\} . a[i] = x$

$v \leftarrow u - l$

assume $a[(l + u)/2] < x$

$l \leftarrow (l + u)/2 + 1$

assume $a[(l + u)/2] \neq x$ **do**

@ $u - l < v \wedge u - l \geq 1 \wedge \exists i \in \{l, \dots, u\} . a[i] = x$

Part of verification condition:

$$[u - l \geq 1 \wedge v = u - l \wedge l_1 = (l + u)/2 + 1] \Rightarrow u - l_1 < v$$

automatic proof (CVC4)

Proof of Verification Condition

$$[u - l \geq 1 \wedge v = u - l \wedge l_1 = \frac{l+u}{2} + 1] \Rightarrow u - l_1 < v$$

Attention: integer division, may have a remainder!

Assume $u - l \geq 1$, $v = u - l$, $l_1 = \frac{l+u}{2} + 1$ and prove $u - l_1 < v$.

It suffices to prove $u - \frac{l+u}{2} + 1 < u - l$.

It suffices to prove that lower bound increases:

$$\frac{l+u}{2} + 1 > l$$

$\frac{l+u}{2} + 1$ as a function in u monotonically increasing.

Proof for $u = l + 1$ suffices: $\frac{l+u}{2} + 1 = \frac{2l+1}{2} + 1 = l + 1 > l$

The basic path and proof of the second branch is similar.

Loop Termination: Proof

First branch of **if-then-else**:

assume $u - l < v \wedge u - l \geq 1 \wedge \exists i \in \{l, \dots, u\} . a[i] = x$

$v \leftarrow u - l$

assume $a[(l + u)/2] < x$

$l \leftarrow (l + u)/2 + 1$

assume $a[(l + u)/2] \neq x$ **do**

@ $u - l < v \wedge u - l \geq 1 \wedge \exists i \in \{l, \dots, u\} . a[i] = x$

Proof outline:

- ▶ If $u - l = 1$, then the last **assume** fails (i.e., the loop terminates).
- ▶ If $u - l = 2$, then, at the final assertion $u - l = 1$.
- ▶ If $u - l > 2$, then the distance will be larger than in the previous case.

Terminology and General Termination Proofs

Proof of program termination: For each loop

- ▶ a **term** t over program variables denoting a function to a set S , and
- ▶ a **well-founded relation** $\prec$ on S s.t.

the value of t decreases for each execution of the loop wrt. $\prec$.

Such a term is called **loop variant** (also *ranking function*)

Corresponding assertions:

$v \leftarrow \infty$

while ... do

 @ $v \succ t \wedge t \in S$

$v \leftarrow t$

 ...

where

- ▶ v is a new, auxiliary variables (for each loop a different one)
- ▶ ∞ is a constant such that for every $x \in S$, $\infty \succ x$.

Connections

Formulation of termination in temporal logic LTL:

$\mathbf{F} \text{ } pc = r,$

where r is the program line with the command **return**.

Variant: similar notion for continuous systems: *Lyapunov function*

Nested Loops

$\text{substr}(a, n, s, l, p) :\Leftrightarrow \forall i \in \{1, \dots, l\} . p + i - 1 \leq n \wedge a[p + i - 1] = s[i]$

Input: $a \in \mathcal{A}, n \in \mathcal{N}, s \in \mathcal{A}, l \in \mathcal{N}$

Output: p , s.t. $\text{substr}(a, n, s, l, p)$, if such a p , exists, and 0, otherwise.

Observation:

for all $a \in \mathcal{A}, n \in \mathcal{N}, s \in \mathcal{A}, l \in \mathcal{N}$

if $p - 1 + l > n$ then not $\text{substr}(a, n, s, l, p)$

Proof:

Let $a \in \mathcal{A}, n \in \mathcal{N}, s \in \mathcal{A}, l \in \mathcal{N}$ be arbitrary, but fixed.

We assume $p - 1 + l > 0$ and $\text{substr}(a, n, s, l, p)$ and derive a contradiction.

Due to the definition of substr , $\forall i \in \{1, \dots, l\} . p + i - 1 \leq n \wedge a[p + i - 1] = s[i]$.

This implies $p + l - 1 \leq n$, which is in contradiction with $p - 1 + l > 0$. $\square$

Nested Loops

@ $a \in \mathcal{A}, n \in \mathcal{N}, s \in \mathcal{A}, l \in \mathcal{N}$

$\bar{p} \leftarrow 0; i \leftarrow 0$

$w_1 \leftarrow -\infty$

while $\neg \text{found} \wedge \bar{p} + l \leq n$ **do**

@ $w_1 < \bar{p} \wedge \bar{p} \leq n - l \wedge \neg \exists p \in \{1, \dots, \bar{p}\} . \text{substr}(a, n, s, l, p)$

$w_1 \leftarrow \bar{p}; \bar{p} \leftarrow \bar{p} + 1; i \leftarrow 1$

while $i \leq l \wedge a[\bar{p} + i - 1] = s[i]$ **do**

$w_2 \leftarrow -\infty$

@ $w_2 < i \wedge i \leq l \wedge \text{substr}(a, n, s, i, \bar{p})$

$w_2 \leftarrow i; i \leftarrow i + 1$

if *found* **then**

@ $\text{substr}(a, n, s, l, \bar{p})$

return $\bar{p}$

else

@ $\neg \exists p . \text{substr}(a, n, s, l, p)$

return 0

Relationship to Program Complexity

If we have a variant for the relation $<$ on natural numbers
we **cannot execute** the corresponding loop **more often** than
the **initial value** of the variant.

So: upper **bound** on the run-time **complexity** of an algorithm

Completeness of the method:

Does there **exist** a variant for **every terminating algorithm**?

Yes, number of loop iterations before termination.

Problem: How to write this as a simple term?

For algorithms whose termination is not known,
we do not know whether this number is finite.

Termination of Recursion

GCD(x, y)

@ $x, y \in \mathcal{N}, x \geq y$

$v \leftarrow y$ // variant y

if $y = 0$ **then**

$r \leftarrow x$

else

@ $v > y[x \leftarrow y, y \leftarrow x \bmod y] \wedge y[x \leftarrow y, y \leftarrow x \bmod y] \geq 0$

$r \leftarrow \text{GCD}(y, x \bmod y)$

return r

well-founded relation? variant? $<$ on $\{0, 1, \dots\}, y$

new verification condition (i.e., extension of old condition):

$$[x \geq y \wedge v = y \wedge y \neq 0] \Rightarrow [x \bmod y < v \wedge x \bmod y \geq 0]$$

Automation

Theorem: [Turing, 1937]:

The problem of verifying termination (halting problem) is **undecidable**.

If we have a variant,

the corresponding verification conditions

can often be proved automatically (e.g., using CVC4)

Can we find variants automatically?

T2 Temporal Prover [Cook et al., 2011]

Rest of lecture: an underlying technique

Automatic Synthesis of Variants

$v \leftarrow \infty$

while $10 \leq x \wedge x \leq 20 \wedge 10 \leq y \wedge y \leq 20$ **do**

 @ $v - 1 \geq ax + by \wedge ax + by \geq 0$

$v \leftarrow ax + by$

$x \leftarrow x - y$

$y \leftarrow x + 2y$

Variant?

Assumption: a linear function

But we do not know which one,

that is, we do not know the **coefficients** of the function.

Variant template: $ax + by$

Well-founded relation $\prec \subseteq \mathbb{R}^{\geq 0} \times \mathbb{R}^{\geq 0}$,

$u \prec v$ iff $u \leq v - 1$.

Automatic Synthesis of Variants: Verification Conditions

$v \leftarrow \infty$

while $10 \leq x \wedge x \leq 20 \wedge 10 \leq y \wedge y \leq 20$ **do**

 @ $v - 1 \geq ax + by \wedge ax + by \geq 0$

$v \leftarrow ax + by$

$x \leftarrow x - y$

$y \leftarrow x + 2y$

Verification conditions (slightly simplified):

$$[10 \leq x \wedge x \leq 20 \wedge 10 \leq y \wedge y \leq 20] \Rightarrow ax + by \geq 0$$

$$\left[\begin{array}{l} ax + by \geq 0 \wedge \\ v = ax + by \wedge \\ x_1 = x - y \wedge y_1 = x + 2y \wedge \\ 10 \leq x_1 \wedge x_1 \leq 20 \wedge 10 \leq y_1 \wedge y_1 \leq 20 \end{array} \right] \Rightarrow [ax_1 + by_1 \leq v - 1 \wedge ax_1 + by_1 \geq 0]$$

Solving the Verification Conditions

$$[10 \leq x \wedge x \leq 20 \wedge 10 \leq y \wedge y \leq 20] \Rightarrow ax + by \geq 0$$

$$\left[\begin{array}{l} ax + by \geq 0 \wedge \\ v = ax + by \wedge \\ x_1 = x - y \wedge y_1 = x + 2y \wedge \\ 10 \leq x_1 \wedge x_1 \leq 20 \wedge 10 \leq y_1 \wedge y_1 \leq 20 \end{array} \right] \Rightarrow [ax_1 + by_1 \leq v - 1 \wedge ax_1 + by_1 \geq 0]$$

We have to find a, b , such that for all x, y, v, x_1, y_1 the conditions hold.

Purely **algebraic problem**, Redlog demo

If the conditions do not have a solution, what does this imply?

This does **not** imply non-termination.

(we only know that there is no variant of the given **form**)

We may use a **different** one, for example,

by increasing the degree of the polynomial: $ax^2 + bcx + cy^2 + ex + fy$

Example:

```
while  $i \neq 0$  do  
  if  $i > 0$  then  
     $i \leftarrow -i$   
  else  
     $i \leftarrow i + 1$ 
```

i
3
-3
-2
-1

Variant?

Lexicographic Relation

Intuition: phone book

Example: 8-bit representation of elements of $\{0, \dots, 255\}$,
 $0 \prec_i 1, i \in \{1, \dots, 8\}$

$$(0, 1, 0, 1, 0, 1, 1, 1) < (0, 1, 0, 1, 1, 0, 0, 0)$$

For given relations $\prec_1, \dots, \prec_n$,

$$(s_1, \dots, s_n) \prec (t_1, \dots, t_n) :\Leftrightarrow \bigvee_{i=1}^n \left(\bigwedge_{j=1}^{i-1} s_j = t_j \wedge s_i \prec_i t_i \right)$$

If $\prec_1, \dots, \prec_n$ are well-founded relations on $S_1, \dots, S_n$, respectively
then $\prec$ is also a well-founded relation on $S_1 \times \dots \times S_n$.

Example:

```
while  $i \neq 0$  do  
  if  $i > 0$  then  
     $i \leftarrow -i$   
  else  
     $i \leftarrow i + 1$ 
```

i	$\text{sgn}(i)$	$ i $
3	1	3
-3	-1	3
-2	-1	2
-1	-1	1

Loop variant: $(\text{sign}(i), |i|)$

$S := \{(s, n) \mid s \in \{-1, 1\}, n \in \mathbb{N}_0\}$.

Well-founded relation over S : lexicographic relation over S wrt. $<$ and $<$.

The loop variant may also contain user-defined functions!

Conclusion

- ▶ **Variants** give evidence for the termination of programs
- ▶ Run-time check of corresponding conditions using **assertions**
- ▶ Termination proofs can be **partially automated**
- ▶ However, in general, **undecidable**

Literature I

Byron Cook, Andreas Podelski, and Andrey Rybalchenko. Proving program termination. *Commun. ACM*, 54(5):88–98, May 2011. doi: 10.1145/1941487.1941509.

A. M. Turing. On computable numbers, with an application to the Entscheidungsproblem. *Proceedings of the London Mathematical Society*, s2-42 (1):230–265, 1937.